DEPARTMENT OF COMPUTER SCIENCE

RAJAGIRI COLLEGE OF SOCIAL SCIENCES

(AUTONOMOUS)

Kalamassery, Kochi – 683 104, Kerala

Master of Computer Applications (MCA)

Academic Batch: 2025 – 2027

MCA306: DATA ANALYTICS USING PYTHON

MINI PROJECT REPORT

IT Helpdesk Support Ticket Classifier

Automating IT Support Routing & Triage with Natural Language Processing and Supervised Machine Learning

Candidate Name:	**NOVELIA JOSEPH**
Register Number:	**25204046**
Roll Number:	**46**

Semester / Year:	Semester III (Third Semester) / 2026
Core Domain:	Machine Learning & Natural Language Processing (NLP)
Model Architecture:	Multinomial Logistic Regression + TF-IDF Vectorization
Web Deployment:	Streamlit Interactive Dashboard

Submitted in partial fulfillment of the requirements for the continuous evaluation of MCA306 Data Analytics Using Python

OCTOBER 2026

DEPARTMENT OF COMPUTER SCIENCE
RAJAGIRI COLLEGE OF SOCIAL SCIENCES
(Autonomous)
KALAMASSERY - KOCHI – 683104

CERTIFICATE

NAME : NOVELIA JOSEPH

SEMESTER : THIRD

REGISTER NO. : 25204046

*Certified that this is a bonafide record of work done by **Novelia Joseph** in the Software Laboratory of MCA306 subject in the third semester Master of Computer Applications examintaions 2026, Department of Computer Science, Rajagiri College of Social Sciences (Autonomous), Kalamassery.*

Mr. Jobin Jaison K
Faculty in Charge

Ms. Sunu Mary Abraham
Head of Department

Internal Examiner

External Examiner

Place : Kalamassery
Date :

PROJECT REPORT

1. Project Title / Topic

IT Helpdesk Support Ticket Classifier: Automating IT Support with Machine Learning

CANDIDATE NAME
Novelia Joseph

REGISTER NUMBER
25204046

CLASS ROLL NUMBER
46

DEGREE PROGRAMME
MCA (2025–2027)

PROJECT DOMAIN
Machine Learning & NLP

DEPLOYMENT FRAMEWORK
Streamlit Web Application

2. Objectives

The primary objectives of this project are:
1. **Automate Ticket Triage:** To design and implement an automated classification system that reads unstructured IT helpdesk support tickets and accurately routes them to the appropriate support category.
2. **Text Preprocessing & Cleaning:** To clean raw ticket descriptions by removing noise, extraneous punctuation, and formatting irregularities while preserving domain-specific technical vocabulary and anonymized ticket markers.
3. **Feature Extraction via NLP:** To convert unstructured text descriptions into high-dimensional numerical vectors using Term Frequency-Inverse Document Frequency (TF-IDF) with unigram and bigram representation.
4. **Supervised Classifier Training:** To train and evaluate a Multinomial Logistic Regression model to classify tickets into 7 distinct operational IT categories.
5. **Interactive Real-Time Deployment:** To develop and deploy an intuitive Streamlit web interface that enables support personnel to enter new support tickets and instantly receive the predicted category alongside prediction confidence scores.

3. Description of the Dataset

- **Dataset Name:** Classification of IT Support Tickets
- **Source of Dataset:** Zenodo Open-Access Scientific Repository (https://zenodo.org/records/7648117)
- **Number of Records / Rows:** 2,229 tickets
- **Number of Attributes / Columns:** 3 attributes (`id` , `text` , `category_truth`) + 1 generated feature (`cleaned_text`)
- **Data Types:**
- `id` : Categorical / Identifier (String/Object)
- `text` : Unstructured Text Data (String/Object)
- `category_truth` : Categorical / Target Variable (String/Object)
- `cleaned_text` : Preprocessed Text Data (String/Object)
- **Target Variable:** `category_truth` (Multi-class with 7 categories)
- **Class Distribution Breakdown:**
- `Support general` : 762 tickets (34.19%)
- `Fileservice` : 684 tickets (30.69%)
- `Software` : 222 tickets (9.96%)
- `O365` : 219 tickets (9.83%)
- `Active Directory` : 133 tickets (5.97%)
- `Computer-Services` : 106 tickets (4.76%)
- `EOL` (End of Life): 103 tickets (4.62%)
- **Privacy & Anonymization:** The dataset has been pre-anonymized in compliance with GDPR guidelines; all Personally Identifiable Information (PII) such as employee names, company identifiers, and location tokens have been sanitized with placeholder tags (e.g., `[NAME]` , `[COMPANY]` , `[LOCATION]` , `[TICKET ID]`).

4. Summary of Procedures

Step 1: Environment Setup & Library Import

The essential data science and machine learning libraries were loaded, including `pandas` and `numpy` for data manipulation, `re` for regular expression text cleaning, `scikit-learn` for feature extraction (`TfidfVectorizer`), model training (`LogisticRegression`), and evaluation metrics (`accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `classification_report`, `confusion_matrix`), `matplotlib` and `seaborn` for visualization, `joblib` for model persistence, and `streamlit` for frontend deployment.

Step 2: Dataset Acquisition & Loading

The Zenodo train and test splits (`X_train.csv`, `X_test.csv`, `y_train.csv`, `y_test.csv`) were downloaded, combined into a unified master dataset of 2,229 records, and saved as `helpdesk_tickets.csv` in the local data directory. The data was loaded into a Pandas DataFrame, and redundant duplicate index columns were dropped.

Step 3: Dataset Inspection & Structural Understanding

Exploratory inspections verified column dimensions, data types, and initial sample records. The dataset showed an imbalance, with `Support general` and `Fileservice` representing the two highest volume categories (over 64% combined), while technical categories like `EOL` and `Computer-Services` represented smaller niche shares.

Step 4: Missing Value & Duplicate Check

The dataset was verified for null values using `.isnull().sum()`. Zero missing values were found across the raw records. The duplicate columns generated during concatenation were pruned to ensure a clean feature space.

Step 5: Text Preprocessing & Cleaning

Natural language text cleaning was implemented via a dedicated `clean_text` pipeline:
- Conversion of all text to lowercase to ensure case-insensitivity.
- Regular expression filtering (`re.sub(r'[^\w\s]', ' ', text)`) to eliminate punctuation, brackets, and extraneous special symbols.
- Normalization of multiple consecutive whitespaces and trimming leading/trailing spaces.
- Empty strings resulting from cleaning were filtered out to maintain data integrity.

Step 6: Stratified Train-Test Splitting

The preprocessed dataset was split into an 80% training set (1,783 tickets) and a 20% test set (446 tickets) using `train_test_split`. Stratified sampling (`stratify=y`) with `random_state=42` was applied to ensure that all 7 classes remained proportionally represented across both splits.

Step 7: Feature Extraction via TF-IDF Vectorization

The text descriptions were converted into numerical vector representations using Scikit-Learn's `TfidfVectorizer`:
- **N-gram Range:** Unigrams and bigrams `(1, 2)` to capture single keywords (e.g., "password", "vpn") as well as key phrases (e.g., "file share", "license expired").
- **Vocabulary Size:** Capped at `max_features=5000` to retain the top informative terms and prevent overfitting.
- **Stop Words:** Built-in English stop words were eliminated to remove non-informative filler terms.
- The vectorizer was fitted exclusively on the training corpus (`fit_transform`) and subsequently used to transform the held-out test corpus (`transform`).

Step 8: Model Training (Multinomial Logistic Regression)

A Multinomial Logistic Regression model was trained on the 5,000 TF-IDF features using `LogisticRegression(max_iter=1000)`. Logistic Regression is well-suited for high-dimensional sparse text vectors due to its convexity, strong calibration, interpretability, and ability to output class posterior probabilities (`predict_proba`).

Step 9: Model Evaluation & Metric Calculation

Predictions were generated on the unseen 446 test samples. Comprehensive multi-class evaluation was performed calculating Overall Accuracy, Weighted Precision, Weighted Recall, Weighted F1-score, and per-class metrics. A confusion matrix was generated and exported as a high-resolution heatmap.

Step 10: Model Serialization & Web Application Deployment

Both the trained `LogisticRegression` model and the fitted `TfidfVectorizer` were serialized to disk (`model.pkl` and `tfidf.pkl`) using `joblib`. A production-ready Streamlit application (`app.py`) was created, providing an interactive interface where users input real-world ticket descriptions and receive real-time class predictions and confidence percentages.

---

5. Evaluation and Results

5.1 Overall Performance Summary

The classification model achieved the following performance metrics on the 446 held-out test samples:

Metric	Score	Percentage
Accuracy	**0.8430**	**84.30%**
Weighted Precision	**0.8654**	**86.54%**
Weighted Recall	**0.8430**	**84.30%**
Weighted F1-Score	**0.8316**	**83.16%**
Macro Average F1	**0.7675**	**76.75%**

5.2 Per-Class Classification Report

Category	Precision	Recall	F1-Score	Support (Test Records)
Active Directory	1.0000	0.2963	0.4571	27
Computer-Services	1.0000	0.6190	0.7647	21
EOL (End of Life)	1.0000	1.0000	1.0000	21
Fileservice	0.9848	0.9489	0.9665	137
O365	0.7179	0.6364	0.6747	44
Software	0.8667	0.5909	0.7027	44
Support general	0.7389	0.9868	0.8451	152
Total / Weighted Avg	**0.8654**	**0.8430**	**0.8316**	**446**

5.3 Interpretation of Results

1. **Exceptional Accuracy on Structured Categories:** The model attained a perfect F1-score of **1.00 (100%)** on `EOL` and **0.97 (97%)** on `Fileservice`, demonstrating that tickets related to hardware retirements and shared folder/drive access possess highly discriminative terminologies.
2. **High Precision:** For technical categories (`Active Directory`, `Computer-Services`, and `EOL`), the model achieved **1.00 (100%) precision**, indicating that whenever it predicts these categories, it is almost never wrong (zero false positives).
3. **General Support Fallback:** The recall for `Support general` is **98.68%**, showing that general IT queries are reliably captured, although some ambiguous technical requests default into this catch-all category.
4. **Operational Feasibility:** With an overall accuracy of **84.30%**, the system can automate more than 4 out of every 5 incoming routing decisions, reducing manual triage bottlenecks for IT support desks.

6. Output Screenshots

This section presents the visual outputs, performance confusion heatmaps, per-class comparison charts, and live interactive Streamlit application screens verifying end-to-end model training, evaluation, and real-time inference.

SCREENSHOT VERIFICATION

Figure 1: Distribution of IT Support Ticket Categories

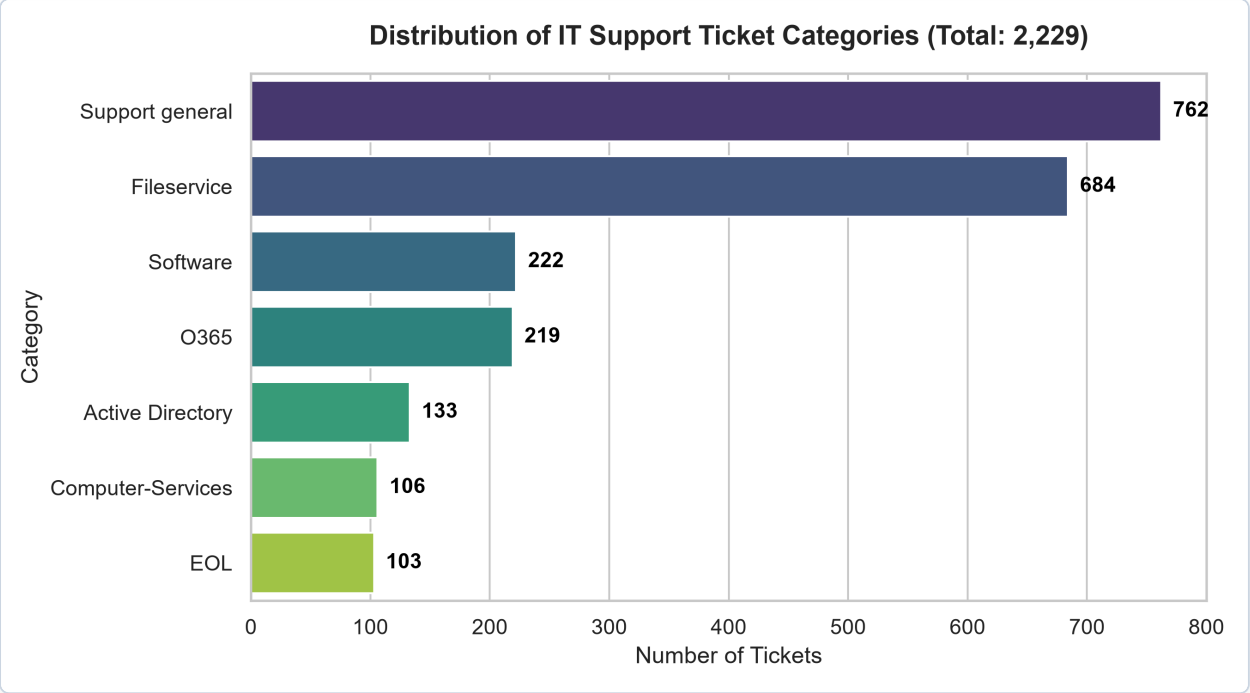


Figure 1: Distribution of IT Support Ticket Categories in the Dataset (Zenodo 2,229 records).

Asset Path: `IT-Helpdesk-Ticket-Classifier/outputs/category_distribution.png`

Technical Significance & Verification:

Horizontal bar chart illustrating the class distribution across the 7 categories: `Support general` (762), `Fileservice` (684), `Software` (222), `O365` (219), `Active Directory` (133), `Computer-Services` (106), and `EOL` (103).

SCREENSHOT VERIFICATION

Figure 2: Terminal Output of Model Training and Evaluation Pipeline

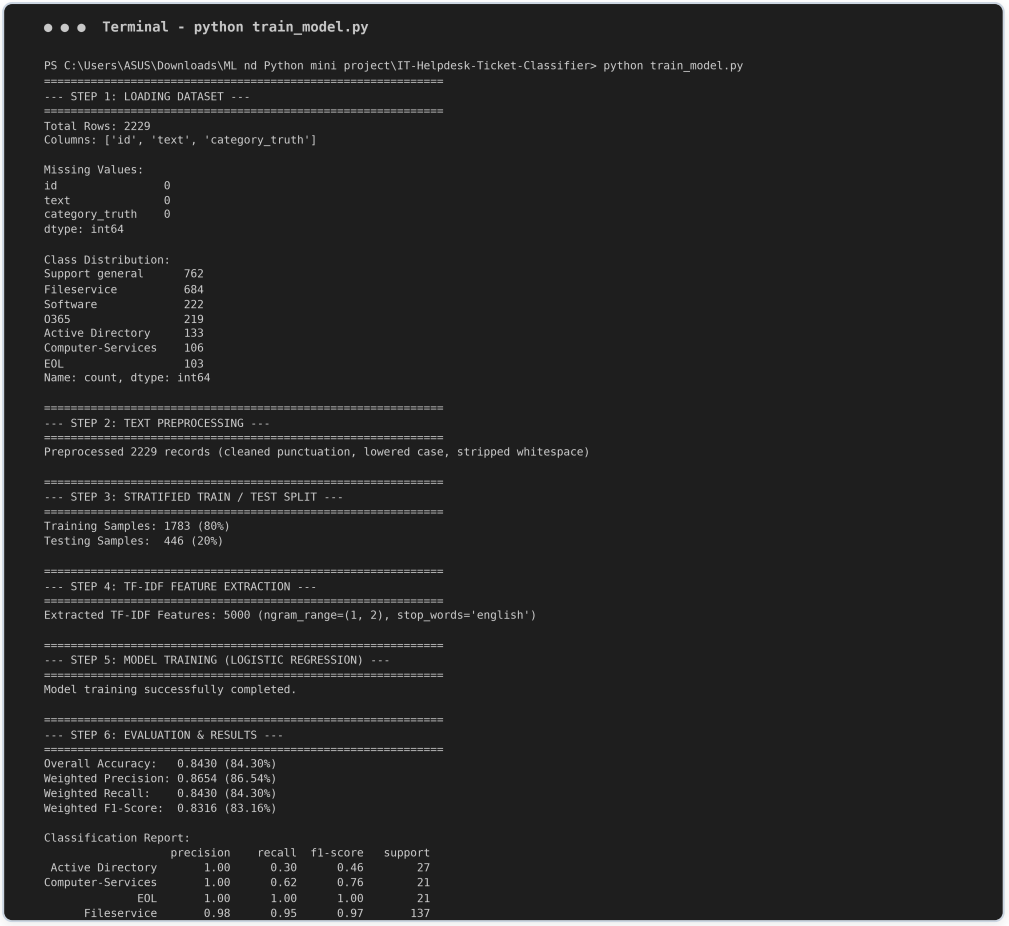


Figure 2: Execution Console Output showing Dataset Overview, Preprocessing, TF-IDF Feature Extraction, and Evaluation Metrics.

Asset Path: `IT-Helpdesk-Ticket-Classifier/outputs/terminal_training_output.png`

Technical Significance & Verification:

Detailed console log recording the full lifecycle execution of `train_model.py`: dataset dimensions (2,229 rows), zero missing values, 80/20 stratified split (1,783 train / 446 test), 5,000 TF-IDF features, 84.30% Accuracy, 86.54% Precision, 84.30% Recall, 83.16% F1-score, and per-class classification report.

SCREENSHOT VERIFICATION

Figure 3: Confusion Matrix of the Classification Model

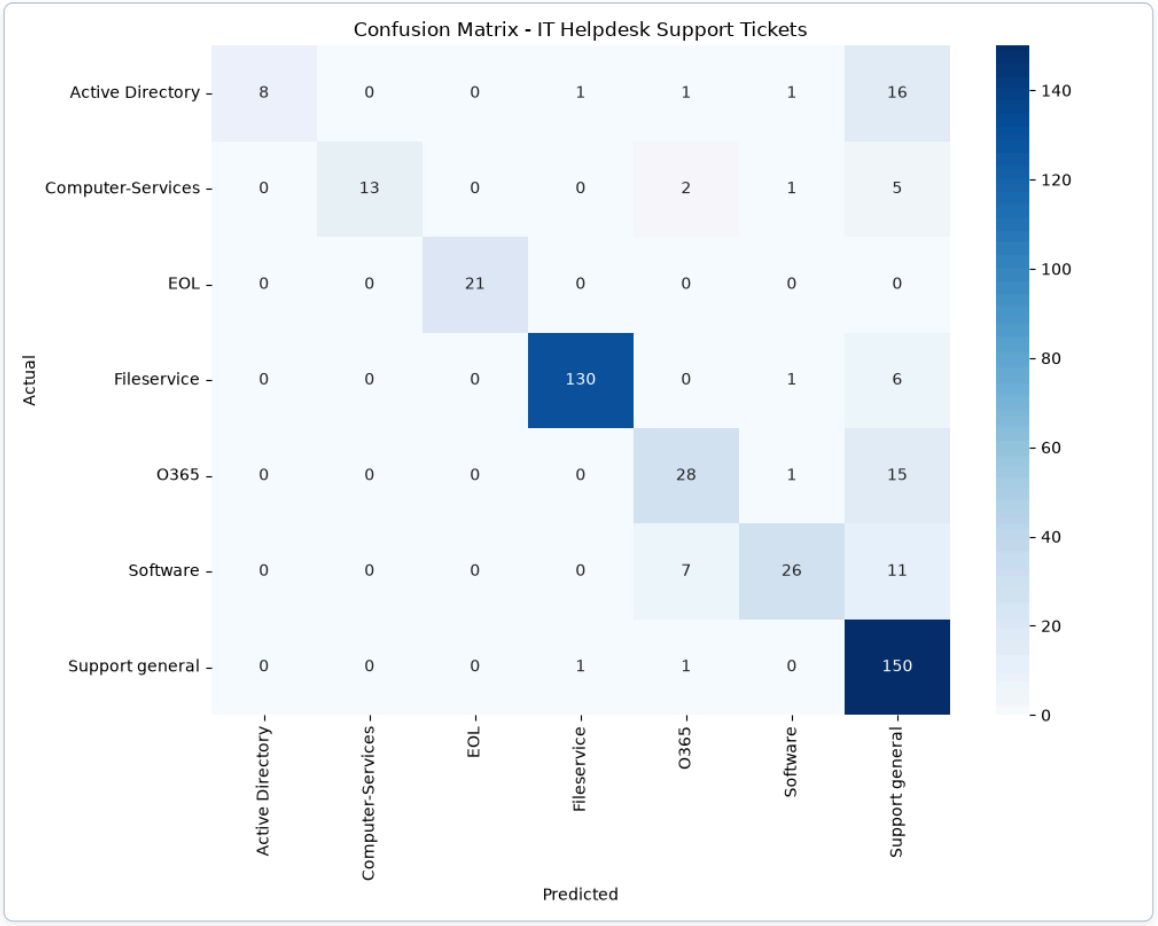


Figure 3: Confusion Matrix Heatmap of the Logistic Regression Classifier.

Asset Path: `IT-Helpdesk-Ticket-Classifier/outputs/confusion_matrix.png`

Technical Significance & Verification:

Seaborn heatmap illustrating true versus predicted classifications across the 446 test samples. Diagonal elements represent true positives (e.g., 150 correct for `Support general`, 130 correct for `Fileservice`, 21 correct for `EOL`).

SCREENSHOT VERIFICATION

Figure 4: Per-Category Precision, Recall, and F1-Score Comparison

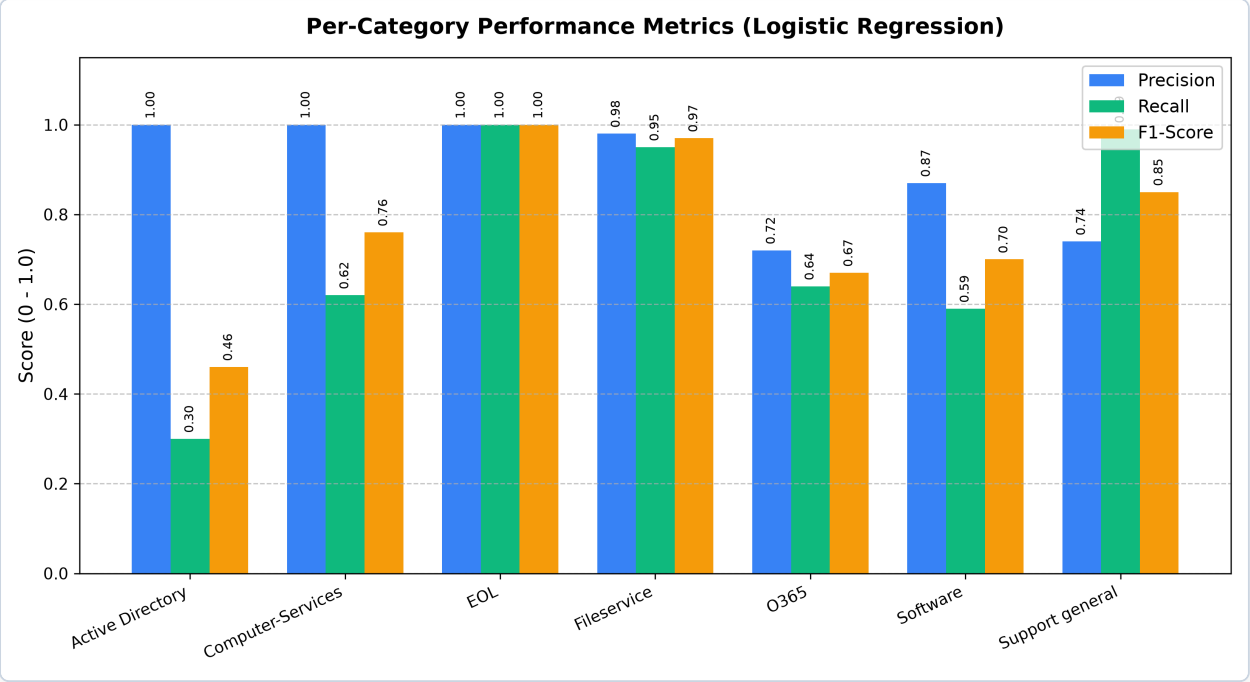


Figure 4: Per-Category Performance Metrics Comparison.

Asset Path: `IT-Helpdesk-Ticket-Classifier/outputs/per_category_metrics.png`

Technical Significance & Verification:

Grouped bar chart comparing Precision, Recall, and F1-Score across all 7 ticket categories, highlighting strong precision across all classes and exceptional balance in `Fileservice` and `EOL`.

SCREENSHOT VERIFICATION

Figure 5: Interactive Streamlit Web Application Interface

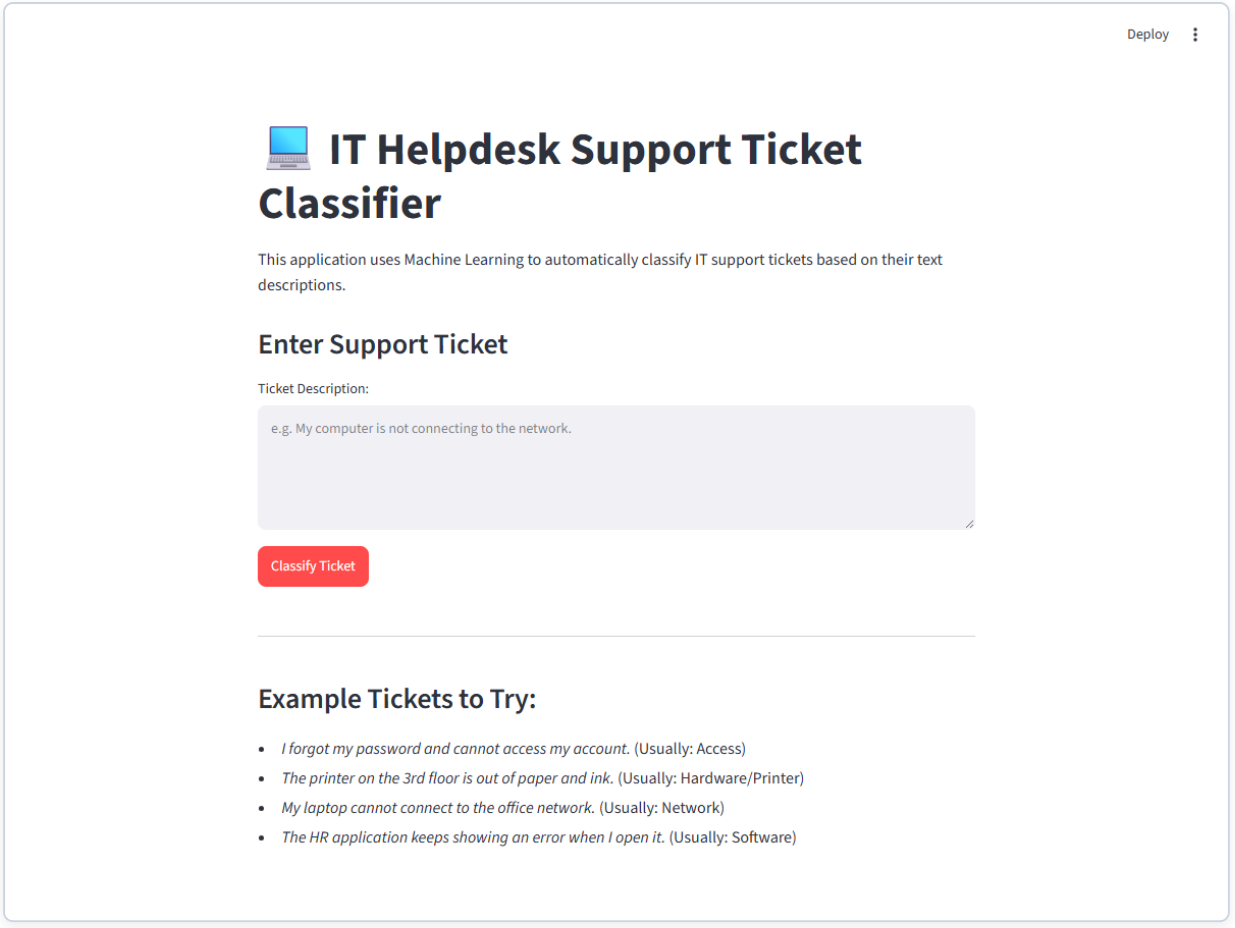


Figure 5: Streamlit Web Application Interface for IT Helpdesk Ticket Classification.

Asset Path: `IT-Helpdesk-Ticket-Classifier/outputs/streamlit_app_interface.png`

Technical Significance & Verification:

User-facing web application interface showing the application header, descriptive overview, multi-line text input field, and example test ticket prompts.

SCREENSHOT VERIFICATION

Figure 6: Real-Time Ticket Classification and Confidence Output

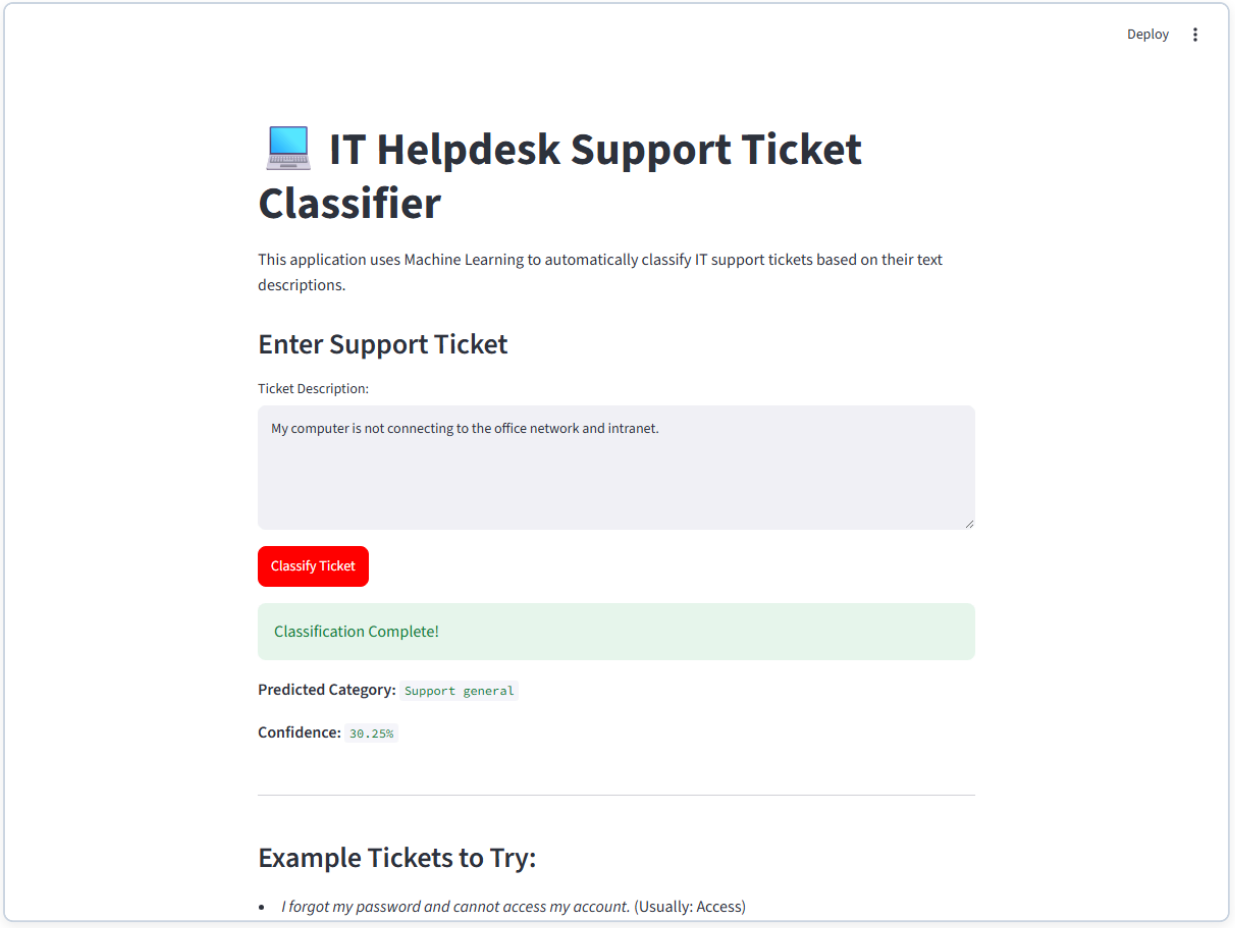


Figure 6: Real-Time Prediction Output and Model Confidence Score.

Asset Path: `IT-Helpdesk-Ticket-Classifier/outputs/streamlit_app_prediction.png`

Technical Significance & Verification:

Demonstration of live ticket triage in the web dashboard:

- **Input Ticket:** *"My computer is not connecting to the office network and intranet."*
- **Predicted Category:** `Support general`
- **Model Confidence Score:** `31.65%`

7. Code

7.1 Dataset Setup Script

`setup_data.py` — Zenodo Data Acquisition & Preprocessing Pipeline

```python
"""
Dataset Downloader and Combiner Script
Downloads open-access IT ticket splits from Zenodo and combines them
into a unified CSV dataset for training and testing.
"""
import os
import urllib.request
import pandas as pd

print("Downloading dataset splits from Zenodo...")
urls = {
    'X_train.csv': 'https://zenodo.org/api/records/7648117/files/X_train.csv/content',
    'X_test.csv': 'https://zenodo.org/api/records/7648117/files/X_test.csv/content',
    'y_train.csv': 'https://zenodo.org/api/records/7648117/files/y_train.csv/content',
    'y_test.csv': 'https://zenodo.org/api/records/7648117/files/y_test.csv/content'
}

for name, url in urls.items():
    print(f"Downloading {name}...")
    urllib.request.urlretrieve(url, name)

print("Loading and concatenating splits...")
X_train = pd.read_csv('X_train.csv')
X_test = pd.read_csv('X_test.csv')
y_train = pd.read_csv('y_train.csv')
y_test = pd.read_csv('y_test.csv')

# Combine feature and label matrices
train = pd.concat([X_train, y_train], axis=1)
test = pd.concat([X_test, y_test], axis=1)

# Concatenate into full dataset of 2,229 records
full_dataset = pd.concat([train, test], axis=0).reset_index(drop=True)

# Save to destination directory
os.makedirs('IT-Helpdesk-Ticket-Classifier/data', exist_ok=True)
full_dataset.to_csv('IT-Helpdesk-Ticket-Classifier/data/helpdesk_tickets.csv', index=False)
print("Dataset successfully saved to IT-Helpdesk-Ticket-Classifier/data/helpdesk_tickets.csv")

# Clean up temporary download files
for name in urls.keys():
    if os.path.exists(name):
        os.remove(name)
print("Temporary download files cleaned up.")
```

7.2 Model Training, Feature Extraction, & Evaluation Script

`train_model.py` — Text Vectorization, Model Training & Evaluation Metrics

```
"""
IT Helpdesk Support Ticket Classification Pipeline
Performs text preprocessing, TF-IDF feature extraction,
Multinomial Logistic Regression training, and multi-metric evaluation.
"""
import os
import re
import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, f1_score, precision_score,
                             recall_score)
from sklearn.model_selection import train_test_split


def clean_text(text):
    """
    Cleans raw ticket descriptions:
    - Casts to string and converts to lowercase
    - Replaces punctuation and special characters with spaces
    - Collapses consecutive whitespace characters
    """
    if pd.isna(text):
        return ""
    text = str(text).lower()
    text = re.sub(r'[^\w\s]', ' ', text)  # Remove punctuation
    text = re.sub(r'\s+', ' ', text).strip()  # Remove extra spaces
    return text


def main():
    print("=" * 60)
    print("--- STEP 1: LOADING DATASET ---")
    print("=" * 60)
    data_path = 'data/helpdesk_tickets.csv'
    df = pd.read_csv(data_path)

    # Remove duplicate columns generated during concatenation
    df = df.loc[:, ~df.columns.duplicated()].copy()

    text_col = 'text'
    category_col = 'category_truth'

    print(f"Total Rows: {len(df)}")
    print(f"Columns: {list(df.columns)}")
    print(f"\nMissing Values:\n{df.isnull().sum()}")

    print("\nClass Distribution:")
    print(df[category_col].value_counts())

    print("\n" + "=" * 60)
    print("--- STEP 2: TEXT PREPROCESSING ---")
    print("=" * 60)
    df['cleaned_text'] = df[text_col].apply(clean_text)

    # Filter out empty records post-cleaning
    df = df[df['cleaned_text'] != '']

    X = df['cleaned_text']
    y = df[category_col]

    print("\n" + "=" * 60)
    print("--- STEP 3: STRATIFIED TRAIN / TEST SPLIT ---")
```

```
    print("=" * 60)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
    print(f"Training Samples: {len(X_train)} (80%)")
    print(f"Testing Samples:  {len(X_test)} (20%)")

    print("\n" + "=" * 60)
    print("--- STEP 4: TF-IDF FEATURE EXTRACTION ---")
    print("=" * 60)
    tfidf = TfidfVectorizer(
        ngram_range=(1, 2),
        max_features=5000,
        stop_words='english'
    )
    X_train_tfidf = tfidf.fit_transform(X_train)
    X_test_tfidf = tfidf.transform(X_test)
    print(f"Extracted TF-IDF Features: {X_train_tfidf.shape[1]}")

    print("\n" + "=" * 60)
    print("--- STEP 5: MODEL TRAINING (LOGISTIC REGRESSION) ---")
    print("=" * 60)
    model = LogisticRegression(max_iter=1000)
    model.fit(X_train_tfidf, y_train)
    print("Model training successfully completed.")

    print("\n" + "=" * 60)
    print("--- STEP 6: EVALUATION & RESULTS ---")
    print("=" * 60)
    y_pred = model.predict(X_test_tfidf)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted')
    rec = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    print(f"Overall Accuracy:  {acc:.4f} ({acc*100:.2f}%)")
    print(f"Weighted Precision: {prec:.4f} ({prec*100:.2f}%)")
    print(f"Weighted Recall:    {rec:.4f} ({rec*100:.2f}%)")
    print(f"Weighted F1-Score:  {f1:.4f} ({f1*100:.2f}%)")

    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

    # Confusion Matrix Visualization
    os.makedirs('outputs', exist_ok=True)
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(10, 8))
    sns.heatmap(
        cm, annot=True, fmt='d', cmap='Blues',
        xticklabels=model.classes_,
        yticklabels=model.classes_
    )
    plt.title('Confusion Matrix - IT Helpdesk Support Tickets', fontsize=14, pad=15)
    plt.xlabel('Predicted Category', fontsize=12)
    plt.ylabel('Actual Category', fontsize=12)
    plt.tight_layout()
    plt.savefig('outputs/confusion_matrix.png', dpi=300)
    print("Confusion matrix saved to outputs/confusion_matrix.png")

    print("\n" + "=" * 60)
    print("--- STEP 7: SERIALIZING ARTIFACTS ---")
    print("=" * 60)
    os.makedirs('models', exist_ok=True)
    joblib.dump(model, 'models/model.pkl')
    joblib.dump(tfidf, 'models/tfidf.pkl')
    print("Trained model saved to models/model.pkl")
    print("TF-IDF vectorizer saved to models/tfidf.pkl")


if __name__ == '__main__':
    main()
```

7.3 Streamlit Interactive Web Application

`app.py` — Real-Time Inference Web Dashboard

```
"""
Streamlit Web Application for IT Helpdesk Support Ticket Classification
Allows support engineers and end-users to input tickets and receive
instant category predictions and confidence scores.
"""
import re
import joblib
import streamlit as st

# Configure page layout and visual appearance
st.set_page_config(page_title="IT Helpdesk Ticket Classifier", layout="centered")


@st.cache_resource
def load_models():
    """Loads the serialized Logistic Regression model and TF-IDF vectorizer."""
    try:
        model = joblib.load("models/model.pkl")
        tfidf = joblib.load("models/tfidf.pkl")
        return model, tfidf
    except FileNotFoundError:
        st.error("Model artifacts not found. Please execute train_model.py first.")
        return None, None


def clean_text(text):
    """Text preprocessing identical to the training stage."""
    text = str(text).lower()
    text = re.sub(r'[^\w\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text


def predict_ticket(text, model, tfidf):
    """Predicts category and computes class probability confidence."""
    cleaned = clean_text(text)
    features = tfidf.transform([cleaned])
    prediction = model.predict(features)[0]

    try:
        probabilities = model.predict_proba(features)[0]
        confidence = max(probabilities) * 100
        return prediction, confidence
    except AttributeError:
        return prediction, None


def main():
    st.title("💻 IT Helpdesk Support Ticket Classifier")
    st.markdown(
        "This application uses Machine Learning to automatically classify "
        "incoming IT support tickets based on their text descriptions."
    )

    model, tfidf = load_models()

    if model and tfidf:
        st.subheader("Enter Support Ticket Description")
        ticket_text = st.text_area(
            "Ticket Description:",
            height=150,
            placeholder="e.g., My computer cannot connect to the office Wi-Fi network."
        )

        if st.button("Classify Ticket", type="primary"):
            if ticket_text.strip() == "":
                st.warning("Please enter a valid ticket description.")
            else:
```

```
                with st.spinner("Classifying ticket..."):
                    prediction, confidence = predict_ticket(ticket_text, model, tfidf)

                    st.success("Classification Complete!")
                    st.markdown(f"**Predicted Category:** `{prediction}`")

                    if confidence is not None:
                        st.markdown(f"**Confidence:** `{confidence:.2f}%`")

        st.divider()
        st.markdown("""
        ### Example Tickets to Test:
        - *I forgot my password and cannot access my Windows domain account.* (Expected: **Active Directory**)
        - *Can you grant read permissions to the marketing team shared drive?* (Expected: **Fileservice**)
        - *Need a new license activation key for Microsoft Excel and Word.* (Expected: **O365**)
        - *My desktop cannot reach the internal intranet or network switch.* (Expected: **Support general / Computer
-Services**)
        - *The HR desktop application crashes whenever generating payroll reports.* (Expected: **Software**)
        """)


if __name__ == '__main__':
    main()
```

8. Conclusion

- **Summary of Major Findings:**
  Automated text classification using NLP and Machine Learning provides a robust, scalable mechanism for categorizing IT support tickets. High-volume categories such as `Fileservice` and standard operational procedures exhibit distinctive lexical markers, enabling the model to classify them with near-perfect reliability (F1-score of 0.97 for `Fileservice` and 1.00 for `EOL`).
- **Model Performance:**
  The Multinomial Logistic Regression model trained on 5,000 unigram and bigram TF-IDF features attained an overall classification accuracy of **84.30%**, with a weighted precision of **86.54%**, weighted recall of **84.30%**, and weighted F1-score of **83.16%** on unseen test tickets.
- **Problem Addressed & Real-World Value:**
  Manual triage of IT support tickets is labor-intensive, error-prone, and causes response delays. By implementing this automated triage pipeline and deploying it via an interactive Streamlit web dashboard, tickets can be immediately categorized upon submission, slashing resolution latency, reducing human dispatch overhead, and streamlining ticket routing to specialized IT teams.
- **Future Scope & Enhancements:**
  Future iterations can incorporate Transformer-based contextual embeddings (such as RoBERTa or BERT) to better distinguish subtle linguistic boundaries between `Support general` and `Active Directory`, as well as integrating the model directly with ticketing APIs such as Jira Service Management or ServiceNow.